

```

"""
CASHFOARE – Equities Asset Class Break Chaining
=====

Identifies root-cause breaks and traces propagation chains
across all 26 equities recs (FO → BO → MO flow).

Usage:
    1. Load your dataframe as `df` (uncomment the pd.read_csv line)
    2. Ensure the column names match the MAP_COLS dict below
    3. Run: python cashfoare_break_chaining.py

Output:
    chain_df    – every break row enriched with BreakChainID + chain metadata
    summary     – one row per chain: root rec, flow path, amounts, rec count
    root_cause  – only the root (first) break from each chain
"""

import pandas as pd
import numpy as np
from collections import defaultdict

# -----
# STEP 0 – LOAD DATA
# -----
# df = pd.read_csv("your_file.csv")
# df = pd.read_parquet("your_file.parquet")    # if using cached Parquet

# -----
# STEP 1 – COLUMN NAME MAPPING
# Adjust these to match your actual column headers
# -----
MAP_COLS = {
    "date":          "Date",
    "entity":        "Entity",
    "asset_class":   "Asset Class",
    "product":       "High Level Product",
    "trade_ref":     "TRADE REF",
    "rec_name":      "Rec Name (as per Rec Cube)",
    "break_amount":  "BREAK AMOUNT GBP",
    "break_id":      "Break ID",              # optional – set to None if not present
    "team":          "Team",                  # optional – set to None if not present
}

# -----

```

```

# STEP 2 - REC PRECEDENCE ORDER
# Reflects the true equities system flow:
#   FO trade capture → BO cash/settlement/accounting → MO P&L/valuation
# All 26 recs from your image are included.
# Recs at the same layer share the same integer bucket so rank_diff
# logic isn't distorted by sub-ordering within a layer.
# _____
REC_ORDER = {

    # — LAYER 1: Front Office - Trade Population & Position _____
    "FOBO Rec-Equities": 1,
    "FOBO Rec-Dividend": 2, # same FO layer
    "Fidessa vs FTP - Stock/Index Futures": 3,
    "Fidessa vs Fiscal vs FTP - Cash Equity": 4,

    # — LAYER 2: Back Office - Cash, Settlement & Contra _____
    "FLARE vs Fiscal Cash Control (Cash Equities)": 10,
    "FLARE vs HUB Cash Contra (EQD Contra/control)": 11,
    "FLARE vs HUB Cash Contra (L&D)": 12,
    "Flare vs SYN - Cash Equities/SBL Control": 13,
    "Netted Cash Control - Cash equities/SBL": 14,

    # — LAYER 3: Back Office - COMPTA Accounting _____
    "CRI COMPTA Rec - SED": 20,
    "CRI COMPTA Rec - SEF": 21,
    "CRI COMPTA Rec - Cash Equities/SBL": 22,

    # — LAYER 4: Middle Office - Valuation / Risk (Sophis vs FTP) _____
    "Sophis vs FTP - Cash Equity (P&L - MTM & Cash)": 30,
    "Sophis vs FTP - EQD (P&L - MTM & Cash)": 31,
    "Sophis vs FTP - ETD PnL": 32,
    "Sophis vs FTP - FX Spot/Fwd/NDF/FX swap PnL": 33,
    "Sophis vs FTP - L&D (P&L - MTM & Cash)": 34,
    "Sophis vs FTP - MTN (P&L - MTM & Cash)": 35,
    "Sophis vs Vault (Murex) - FXO book level": 36,

    # — LAYER 5: Middle Office - Funding / HLR _____
    "Global1 vs FTP ( HLR - SBL/FCT)": 40,
    "HLR - Prime Services": 41,

    # — LAYER 6: Middle Office - P&L Recs (ACT sign-off) _____
    "P&L Rec - CEQ": 50,
    "P&L Rec - SED": 51,
    "P&L Rec - SEF": 52,

    # — LAYER 7: Fiscal / Washbook (terminal) _____
    "Washbook '0006 Fiscal account' - Market Charges": 60,

```

```

}

# Layer band boundaries – used for chain validation
LAYER_BANDS = {
    "FO": (1, 4),
    "BO_CASH": (10, 14),
    "BO_ACCT": (20, 22),
    "MO_VAL": (30, 36),
    "MO_HLR": (40, 41),
    "MO_PL": (50, 52),
    "FISCAL": (60, 60),
}

# -----
# STEP 3 – STANDARDISE DATA
# -----

def standardise(df: pd.DataFrame, col_map: dict) -> pd.DataFrame:
    """Normalise column types and fill nulls."""
    df = df.copy()

    # Date
    df[col_map["date"]] = pd.to_datetime(df[col_map["date"]], errors="coerce")

    # String columns
    str_cols = ["entity", "asset_class", "product", "trade_ref", "rec_name"]
    for key in str_cols:
        col = col_map[key]
        if col in df.columns:
            df[col] = df[col].astype(str).str.strip().fillna("UNKNOWN")

    # Amount
    amt_col = col_map["break_amount"]
    df[amt_col] = pd.to_numeric(df[amt_col], errors="coerce").fillna(0)

    # Optional columns
    for key in ["break_id", "team"]:
        col = col_map.get(key)
        if col and col in df.columns:
            df[col] = df[col].astype(str).fillna("")

    return df

df = standardise(df, MAP_COLS)

# -----
# STEP 4 – REC RANK + LAYER ASSIGNMENT

```

```

# -----
rec_col = MAP_COLS["rec_name"]

df["rec_rank"] = df[rec_col].map(REC_ORDER)

# Recs not in the order dict get rank 999 (unknown – placed at end)
unknown_recs = df.loc[df["rec_rank"].isna(), rec_col].unique()
if len(unknown_recs) > 0:
    print(f"[WARNING] {len(unknown_recs)} rec(s) not in REC_ORDER – assigned rank")
    for r in unknown_recs:
        print(f"          '{r}'")

df["rec_rank"] = df["rec_rank"].fillna(999).astype(int)

# Assign layer label for reporting
def get_layer(rank: int) -> str:
    for name, (lo, hi) in LAYER_BANDS.items():
        if lo <= rank <= hi:
            return name
    return "UNKNOWN"

df["rec_layer"] = df["rec_rank"].apply(get_layer)

# -----
# STEP 5 – TRADE KEY CONSTRUCTION
# Two keys:
#   TradeKey          – exact match (same trade ref across systems)
#   TradeKey_Fuzzy    – amount-bucketed fallback (cross-system same trade)
# -----

entity_col = MAP_COLS["entity"]
ac_col     = MAP_COLS["asset_class"]
prod_col   = MAP_COLS["product"]
ref_col    = MAP_COLS["trade_ref"]
amt_col    = MAP_COLS["break_amount"]

df["TradeKey"] = (
    df[entity_col] + "|" +
    df[ac_col]     + "|" +
    df[prod_col]   + "|" +
    df[ref_col]
)

# Fuzzy key: round amount to nearest £1,000 – tolerates minor system differences
df["amt_bucket"] = (df[amt_col].abs() / 1000).round(0).astype(int).astype(str)

df["TradeKey_Fuzzy"] = (
    df[entity_col] + "|" +

```

```

        df[ac_col]      + "|" +
        df[prod_col]    + "|" +
        df["amt_bucket"]
    )

# -----
# STEP 6 – CHAINING PARAMETERS
# These govern when two break rows are considered part of the same chain.
# Tune based on your data's settlement cycle and reporting frequency.
# -----
MAX_DATE_GAP_DAYS      = 5      # T+5: covers standard settlement + 1 buffer day
MAX_ROOT_RANK_SPAN     = 59     # max rank spread from chain root (FO=1 to Washbook=60
MONTH_END_GAP_DAYS     = 35     # month-end carry-forward window (crosses month bounda
MIN_FORWARD_STEP       = 0      # rec_rank must not go backwards within a chain

# -----
# STEP 7 – CORE CHAINING ALGORITHM
# Logic:
#   - Group by TradeKey
#   - Within each group, sort by (Date, rec_rank)
#   - Chain rows together when:
#       (a) Date gap is within threshold (normal or month-end carry)
#       (b) rec_rank is non-decreasing (always forward in system flow)
#       (c) Total rank span from chain root is within MAX_ROOT_RANK_SPAN
# -----
date_col = MAP_COLS["date"]

def is_month_end_carry(row_date: pd.Timestamp, last_date: pd.Timestamp) -> bool:
    """True if the gap crosses a month boundary and is within the carry window."""
    gap = (row_date - last_date).days
    return (
        gap <= MONTH_END_GAP_DAYS and
        row_date.month != last_date.month
    )

def chain_group(group: pd.DataFrame, chain_id_start: int) -> tuple[list, int]:
    """
    Apply chaining logic to a single TradeKey group.
    Returns list of enriched row dicts and the next chain_id counter.
    """
    group = group.sort_values([date_col, "rec_rank"]).reset_index(drop=True)
    linked_rows = []
    chain_id = chain_id_start

    current_chain: list[pd.Series] = []
    chain_root_rank: int = 0

```

```

last_date: pd.Timestamp = pd.NaT

def close_chain(chain: list, cid: int) -> list:
    """Stamp all rows in a chain with the chain ID and root metadata."""
    root = chain[0]
    root_rec = root[rec_col]
    root_rank = root["rec_rank"]
    root_date = root[date_col]
    enriched = []
    for r in chain:
        row_dict = r.to_dict()
        row_dict["BreakChainID"] = cid
        row_dict["ChainRootRec"] = root_rec
        row_dict["ChainRootRank"] = root_rank
        row_dict["ChainRootDate"] = root_date
        row_dict["ChainPosition"] = chain.index(r) + 1
        row_dict["ChainLength"] = len(chain)
        row_dict["IsRootBreak"] = (chain.index(r) == 0)
        enriched.append(row_dict)
    return enriched

for _, row in group.iterrows():

    if not current_chain:
        # Start first chain
        current_chain = [row]
        chain_root_rank = row["rec_rank"]
        last_date = row[date_col]
        continue

    # — Gate 1: date continuity —————
    date_gap = (row[date_col] - last_date).days
    normal_window = date_gap <= MAX_DATE_GAP_DAYS
    month_end_window = is_month_end_carry(row[date_col], last_date)
    date_ok = normal_window or month_end_window

    # — Gate 2: forward flow (rec_rank non-decreasing) —————
    step_rank_diff = row["rec_rank"] - current_chain[-1]["rec_rank"]
    forward_ok = step_rank_diff >= MIN_FORWARD_STEP

    # — Gate 3: root span (total reach from chain start) —————
    root_span = row["rec_rank"] - chain_root_rank
    span_ok = root_span <= MAX_ROOT_RANK_SPAN

    # — Decision —————
    if date_ok and forward_ok and span_ok:
        current_chain.append(row)

```

```

        last_date = row[date_col]
        # Note: chain_root_rank stays fixed at the first row's rank

    else:
        # Close current chain and start a new one
        chain_id += 1
        linked_rows.extend(close_chain(current_chain, chain_id))

        current_chain = [row]
        chain_root_rank = row["rec_rank"]
        last_date = row[date_col]

    # Close the final chain
    if current_chain:
        chain_id += 1
        linked_rows.extend(close_chain(current_chain, chain_id))

    return linked_rows, chain_id

# — Run chaining on exact TradeKey first —————
print("Running exact TradeKey chaining...")
all_linked: list = []
global_chain_id = 0

df_sorted = df.sort_values([date_col, "rec_rank"])

for trade_key, group in df_sorted.groupby("TradeKey"):
    rows, global_chain_id = chain_group(group, global_chain_id)
    all_linked.extend(rows)

chain_df = pd.DataFrame(all_linked)

# —————
# STEP 8 – FUZZY PASS
# For breaks that ended up as singleton chains (no links found via TradeKey),
# attempt to link using TradeKey_Fuzzy (entity + product + £ bucket).
# This catches cross-system breaks where trade ref differs.
# —————
print("Running fuzzy TradeKey pass for singleton chains...")

singleton_ids = (
    chain_df.groupby("BreakChainID")["BreakChainID"]
    .count()
    .loc[lambda x: x == 1]
    .index
)

```

```

singleton_mask = chain_df["BreakChainID"].isin(singleton_ids)
singletons      = df.loc[df.index.isin(chain_df.loc[singleton_mask].index)]

fuzzy_linked: list = []
for fuzzy_key, group in singletons.sort_values([date_col, "rec_rank"]).groupby("T
    if len(group) > 1:
        rows, global_chain_id = chain_group(group, global_chain_id)
        fuzzy_linked.extend(rows)

# Merge fuzzy results back – replace singleton rows that got linked
if fuzzy_linked:
    fuzzy_df = pd.DataFrame(fuzzy_linked)
    newly_linked_idx = fuzzy_df.index
    chain_df = pd.concat(
        [chain_df.loc[~chain_df.index.isin(newly_linked_idx)], fuzzy_df],
        ignore_index=True,
    )
    print(f" Fuzzy pass linked {len(fuzzy_df)} additional breaks into chains.")
else:
    print(" No additional links found via fuzzy pass.")

# -----
# STEP 9 – SUMMARY TABLE
# One row per BreakChainID with root cause, rec flow path, amounts.
# Sort chain_df by rec_rank BEFORE aggregating to ensure Rec_Flow is ordered.
# -----
chain_df_ordered = chain_df.sort_values(["BreakChainID", date_col, "rec_rank"])

def ordered_unique(series: pd.Series) -> list:
    """Deduplicated list preserving first-seen order."""
    seen = []
    for v in series:
        if v not in seen:
            seen.append(v)
    return seen

def rec_flow_str(series: pd.Series) -> str:
    """Build ordered unique rec flow string."""
    return " → ".join(ordered_unique(series))

def layer_flow_str(series: pd.Series) -> str:
    return " → ".join(ordered_unique(series))

summary = (
    chain_df_ordered.groupby("BreakChainID")
    .agg(
        TradeKey          = ("TradeKey",          "first"),

```



```

        Trade_Ref      = (ref_col,      "first"),
        Entity         = (entity_col,   "first"),
        Asset_Class    = (ac_col,       "first"),
        Product        = (prod_col,     "first"),

        Root_Rec       = ("ChainRootRec", "first"),
        Root_Rank       = ("ChainRootRank", "first"),
        Root_Layer     = ("rec_layer",    "first"),
        Root_Date       = ("ChainRootDate", "first"),

        Start_Date     = (date_col,     "min"),
        End_Date        = (date_col,     "max"),
        Duration_Days   = (date_col,     lambda x: (x.max() - x.min()).days

        Rec_Count      = (rec_col,      "nunique"),
        Rec_List        = (rec_col,      lambda x: ordered_unique(x)),
        Rec_Flow        = (rec_col,      rec_flow_str),
        Layer_Flow      = ("rec_layer",  layer_flow_str),

        Total_Breaks    = (date_col,     "count"),
        Total_Amount_GBP = (amt_col,     "sum"),
        Max_Break_Amount = (amt_col,     "max"),
        Min_Break_Amount = (amt_col,     "min"),
    )
    .reset_index()
)

# -----
# STEP 10 – DERIVED FLAGS
# -----

# Is this a genuine multi-rec cascade (the interesting ones for PoC)?
summary["Is_Cascade"]      = summary["Rec_Count"] > 1

# Crosses a layer boundary (FO→BO or BO→MO)?
summary["Crosses_Layer"]   = summary["Layer_Flow"].str.contains("→")

# Root cause is in the FO layer?
summary["Root_Is_FO"]      = summary["Root_Rank"] <= 4

# Chain spans all three layers?
summary["Full_Stack_Chain"] = (
    summary["Rec_Flow"].str.contains("FOBO|Fidessa") &
    summary["Rec_Flow"].str.contains("FLARE|Netted|COMPTA") &
    summary["Rec_Flow"].str.contains("Sophis|P&L Rec|Washbook")
)

```

```

# High value chain
summary["High_Value_Chain"] = summary["Total_Amount_GBP"].abs() >= 1_000_000

# -----
# STEP 11 – ROOT CAUSE TABLE
# Isolate just the root (first) break of each chain.
# This is the break the analyst needs to investigate – all others are inherited.
# -----
root_cause = chain_df_ordered[chain_df_ordered["IsRootBreak"]].copy()

root_cause = root_cause.merge(
    summary[[
        "BreakChainID", "Rec_Count", "Rec_Flow", "Layer_Flow",
        "Total_Breaks", "Total_Amount_GBP", "Is_Cascade",
        "Full_Stack_Chain", "High_Value_Chain",
    ]],
    on="BreakChainID",
    how="left",
)

root_cause["Downstream_Breaks"] = root_cause["Total_Breaks"] - 1

root_cause = root_cause.sort_values("Total_Amount_GBP", ascending=False)

# -----
# STEP 12 – PRODUCT-LEVEL CHAIN ANALYSIS
# Shows which product (CEQ / LnD / MTN / OTC) drives the most cascades.
# -----
product_summary = (
    summary[summary["Is_Cascade"]]
    .groupby("Product")
    .agg(
        Total_Chains = ("BreakChainID", "count"),
        Avg_Recs_Per_Chain = ("Rec_Count", "mean"),
        Max_Recs_In_Chain = ("Rec_Count", "max"),
        Total_Amount_GBP = ("Total_Amount_GBP", "sum"),
        FO_Root_Chains = ("Root_Is_FO", "sum"),
        Full_Stack_Chains = ("Full_Stack_Chain", "sum"),
    )
    .reset_index()
    .sort_values("Total_Chains", ascending=False)
)

# -----
# STEP 13 – REC-LEVEL ROOT CAUSE FREQUENCY
# Shows which recs most commonly act as root causes.
# -----

```

```

rec_root_freq = (
    summary[summary["Is_Cascade"]]
    .groupby("Root_Rec")
    .agg(
        Times_Root      = ("BreakChainID", "count"),
        Avg_Chain_Length = ("Rec_Count",    "mean"),
        Total_Amount_GBP = ("Total_Amount_GBP", "sum"),
    )
    .reset_index()
    .sort_values("Times_Root", ascending=False)
)

rec_root_freq["Root_Rec_Rank"] = rec_root_freq["Root_Rec"].map(REC_ORDER).fillna(
rec_root_freq = rec_root_freq.sort_values("Times_Root", ascending=False)

# -----
# STEP 14 - PRINT DIAGNOSTICS
# -----

print("\n" + "="*60)
print("CASHFOARE BREAK CHAINING - RESULTS")
print("="*60)

total_breaks = len(chain_df)
total_chains = summary["BreakChainID"].nunique()
cascade_count = summary["Is_Cascade"].sum()
singleton_count = total_chains - cascade_count

print(f"\nTotal breaks processed : {total_breaks:,}")
print(f"Total chains formed      : {total_chains:,}")
print(f"  ↳ Cascade chains        : {cascade_count:,}   ({cascade_count/total_chains*100:.1f}%)")
print(f"  ↳ Singleton breaks      : {singleton_count:,}   ({singleton_count/total_chains*100:.1f}%)")
print(f"  ↳ Full-stack chains     : {summary['Full_Stack_Chain'].sum():,}")
print(f"  ↳ High-value chains     : {summary['High_Value_Chain'].sum():,}   (≥ £1M)")

print("\n— Top 10 Root Cause Recs —")
print(rec_root_freq.head(10).to_string(index=False))

print("\n— Product Cascade Summary —")
print(product_summary.to_string(index=False))

print("\n— Sample Chain Summary (top 5 by amount) —")
cols_to_show = [
    "BreakChainID", "Product", "Root_Rec", "Rec_Flow",
    "Rec_Count", "Total_Amount_GBP", "Duration_Days",
]
print(summary.nlargest(5, "Total_Amount_GBP")[cols_to_show].to_string(index=False)

# -----

```

```
# STEP 15 - EXPORT
# -----
# chain_df_ordered.to_csv("break_chain_detailed.csv", index=False)
# summary.to_csv("break_chain_summary.csv", index=False)
# root_cause.to_csv("break_chain_root_causes.csv", index=False)
# product_summary.to_csv("product_cascade_summary.csv", index=False)
# rec_root_freq.to_csv("rec_root_cause_frequency.csv", index=False)

print("\n[DONE] Outputs ready: chain_df_ordered, summary, root_cause, product_sum
```